

A WALKTHROUGH FOR STUDENTS

How an AI coding agent actually works

Inside **llmc-code** — a coding assistant that runs on your own laptop. No cloud. No API bill. We open every part and explain it in plain words.

~13,000 lines of Python

26 modules

1,355 tests

3 libraries

100% offline

Every slide answers three things — WHY we needed it · WHAT it is · HOW it works

01 The problem we are solving

WHY THIS EXISTS

You can download a great model today. You still can't use it to code.

A language model does exactly one thing: **you give it text, it gives you text back.** That's all. It cannot open a file. It cannot run your tests. It forgets everything the moment it replies.

So when you load a model in LM Studio, you get a **chat box**. What you actually wanted was something that reads your project, changes your code, runs your tests, sees what broke, and fixes it.

BUT RUNNING LOCALLY CHANGES EVERYTHING

A cloud tool can assume a huge, fast, well-behaved model. **We can't assume any of that.** Our model is smaller, slower, and messier. Almost every custom part in this deck exists because of that one difference.

THE MISSING PIECE

Something has to sit **between the model and your computer**. It gives the model a way to ask for things, does those things for it, and shows it what happened.

That middle program is called a **harness**. Building one is what this deck is about.

Why run it on your own machine?

- **Your code stays yours.** Nothing is uploaded anywhere.
- **It's free to use.** No per-word charges, no limits.
- **It works with no internet.** On a plane, in a lab, anywhere.

Five things local models force you to build

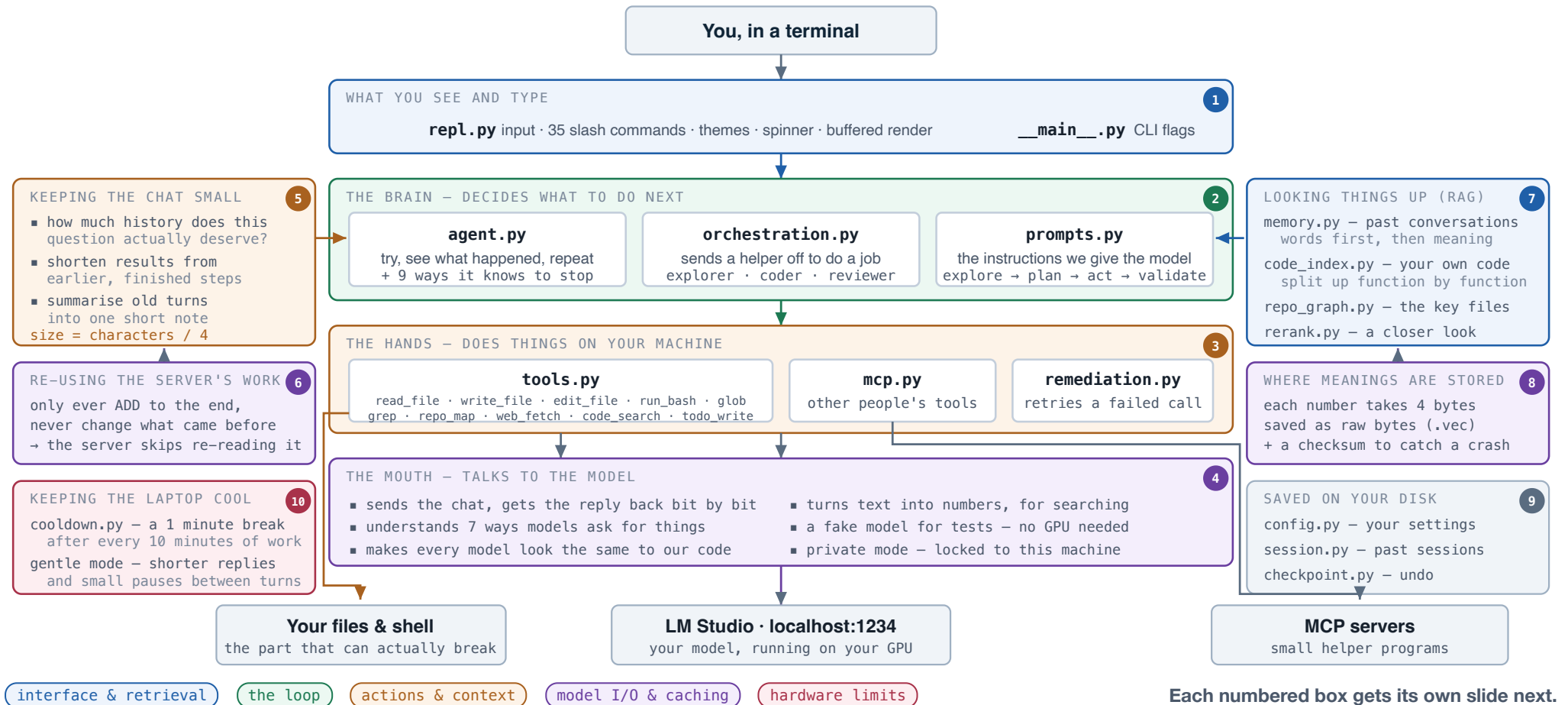
THE DIFFICULTY	WHAT WE HAD TO BUILD
Every model asks for things differently	7 different readers for the same request, plus a layer that fixes wrong spellings of the same option name
Long chats get very slow the model re-reads everything each time	Context management — keep the conversation small, and re-use the work the server already did
The model can only hold a little text at once	Search — store everything, then look up only the few bits this question needs
Laptops get hot	Forced breaks and shorter replies , so an hour of work doesn't cook the machine
It changes your real files	Ask-before-you-act, undo, and limits on where it can write

IN ONE SENTENCE

We are building the program that turns "a model on your laptop" into "an assistant that gets work done on your laptop."

02 The complete architecture

EVERY MODULE, ONE PICTURE



WHY IT IS BUILT IN LAYERS

The brain **never touches your files or the network itself**. It can only ask the hands and the mouth to do it. That one rule is what lets us swap the real model for a fake one and test the whole thing without a GPU.

WHAT THE SIDE PANELS DO

Left: keep it fast and keep the laptop cool. **Right:** help it remember and look things up. A cloud tool needs neither — both exist purely because this runs on your own machine.

HOW TO READ IT

Go **top to bottom** for one question. The panels on either side are helpers the brain calls *before* and *around* each trip to the model. Only the bottom row is not our code.

repl.py

The prompt, the commands, the colours, the status bar.

__main__.py

Command-line flags, and the one-shot mode.

config.py

Your settings.

session.py

Past conversations.

checkpoint.py

Undo.

Why is this 4,000 lines?

Because two libraries both want to control the same terminal: one owns **typing** (history, editing, tab-completion), the other owns **colour and layout**. Left alone they fight over the cursor, and you get half-drawn prompts and stray characters. Most of that file is keeping them out of each other's way.

Two decisions worth copying

DECISION	WHY
Don't print the reply as it arrives — collect it, then print once	Printed live, headings and code blocks show up as raw <code>##</code> and backticks. Collected first, they render properly. The spinner is what moves while you wait.
The status bar reads a cached value , refreshed once per turn	It's redrawn on <i>every keystroke</i> . Asking git for the branch name that often would make typing visibly laggy. Expensive work happens once a turn, not once a letter.

Why a whole box just for saving things?

Because all three have the same failure mode: **a crash halfway through writing**. Settings, conversations and undo copies are each one file that must be either completely old or completely new — never half-written.

THE TRICK, USED IDENTICALLY IN ALL THREE

Never write to the real file. Write to a **temporary file next to it**, then rename it over the top. Renaming is a single, indivisible step — so a crash leaves you with either the old file or the new one, and never a broken half of each.

Settings: the rule that stops surprises

Three layers, in order: **built-in defaults**, then **your saved file**, then **flags you typed just now**. Each layer only overrides what it actually mentions. And a flag you typed is **not saved** unless you ask for it — so a quick experiment never quietly becomes your permanent setup.

WHY THIS MATTERS MORE WHEN IT'S LOCAL

The saved conversation contains **the contents of files the model read**. On a cloud tool that would be a real problem. Here it never leaves the machine — which is exactly the reason people run it locally in the first place.

AND THE ONE THAT STOPS IT ANNOYING YOU

A line starting with `/` is only treated as a command if it's one we actually know. Anything else is sent to the model as ordinary text — so asking about *another* tool's `/deploy` command doesn't get hijacked.

agent.py

Runs the loop. Holds the conversation. Decides when to stop.

orchestration.py

Sends a second assistant off to do a job on its own.

prompts.py

The standing instructions that shape how it behaves.

What does each one actually do?

agent.py keeps a plain Python list of messages. Each round it sends that list to the model. If the reply contains a request, it runs it, adds the result to the list, and goes round again. If the reply is just words, that's the answer — stop.

orchestration.py adds one special tool: "start another assistant". That helper gets a fresh, empty conversation and a smaller set of abilities. It reports back in a few lines.

prompts.py holds the instructions for four different jobs — planner, explorer, coder, reviewer — that get pasted at the top of the conversation.

How do the three fit together?

Three separate questions, three separate files:

- **prompts.py** decides **HOW** it behaves.
- **agent.py** decides **WHEN** to stop.
- **orchestration.py** decides **WHO** does the work.

Change one and the other two keep working — which is why the behaviour can be tuned by editing English, with no code change at all.

THE RULE THAT MAKES IT TESTABLE

The brain **never opens a file and never opens a connection**. It only ever asks the hands or the mouth. So we can hand it a fake model and a temporary folder and test the entire decision-making layer — which is how 1,355 tests run offline in seconds.

Why does a *local* model need all this?

A big cloud model mostly gets it right first time. A small local one does not, and the brain is where you absorb that:

WHAT GOES WRONG LOCALLY	WHAT THE BRAIN DOES ABOUT IT
It forgets to look first and answers from imagination	<code>prompts.py</code> spells out <i>explore</i> → <i>plan</i> → <i>act</i> → <i>validate</i> , and caps how many files it may read before it must search instead
It loops — asking for the same file over and over	<code>agent.py</code> compares each round's requests to the last. Four identical rounds and it stops
It sends a broken request	Three rounds where <i>everything</i> failed to parse and it stops. One good request resets the count — progress is progress
It thinks, then says nothing	Re-asks up to twice, with the reply length cap lifted
A big job won't fit in its small memory	<code>orchestration.py</code> splits it: helpers do the reading, the main conversation only receives conclusions

THE SHAPE TO COPY

Your loop should assume the model **will** misbehave. Every stop-condition here exists because something actually went wrong once — and each one has a **number** attached, so it can never argue with you about when to give up.

tools.py

The 11 things the model is allowed to do, and every safety check.

mcp.py

Tools other people wrote, plugged in at runtime.

remediation.py

Fixes an obviously-wrong request and quietly tries again.

Why is this one box, and not scattered everywhere?

This is the **only** place the model can affect the real world. One box means every dangerous thing goes through the same door — so the confirmation prompt, the folder limit, the output cap and the secret-stripping are written **once**.

If file writing lived in three places, you'd have three chances to forget the check.

What every tool must promise

Three rules, no exceptions:

RULE	WHY IT HAS TO BE A RULE
Never crash	A crash ends the whole turn. A failure <i>message</i> gets read by the model, which then tries something else.
Always cut output short	Whatever a tool returns is added to the conversation and re-sent <i>forever after</i> . One giant result poisons the rest of the session.
Explain the failure	<code>"error"</code> makes the model guess. <code>"no such path; did you mean llmcode/agent.py?"</code> gets fixed next round.

YOUR ERROR MESSAGES ARE NOW PROMPTS

The most surprising thing about building an agent: the text you put in an error is **read by the model and acted on**. Write it for the model, not for a log file.

How does a tool actually get used?

Four steps, every time:

#	STEP
1	Its description goes to the model with the conversation — name, purpose, options.
2	The model replies asking for it by name, with options filled in.
3	We fix the option names (local models get them wrong), ask you if it's dangerous, then run ordinary Python .
4	The result is cut short and added to the conversation.

Why it matters more on a local model

A small model gets paths and arguments wrong **constantly**. So the hands do two extra jobs a cloud harness wouldn't bother with:

- **remediation.py** corrects the obvious mistakes and retries, instead of burning a whole round on a typo.
- **The caps are tighter** than you would set for a big model — the memory it all has to fit into is much smaller.

AND ABOUT OTHER PEOPLE'S TOOLS

An **mcp.py** tool can declare "I'm harmless, don't ask". A bad one could say that about something destructive — so the claim is **ignored** unless you said you trust that program. **Trust is your call, never the tool's**.

the real one

Sends the conversation to LM Studio and reads the reply back as it arrives.

the translator

Understands 7 ways of asking for things, and turns them all into one shape.

the fake one

A scripted stand-in model, so the tests need no GPU and no internet.

Why is there a box here at all?

Because the rest of the program should **never know which model you loaded**. Every family has its own quirks; if those leaked into the brain, changing model would mean changing the loop. So this box always emits the **same three things**:

real model or fake one – both produce only these

```
{"type": "text",      "text": ... a piece of the reply ...}
{"type": "tool_call", "name", "arguments": ... what it wants done ...}
{"type": "done",      # exactly one, always last}
```

The fiddly bit: replies arrive in pieces

A single request arrives **split across dozens of fragments** — and if the model asks for two things at once, the fragments are **interleaved**. We collect them into numbered slots and assemble them only once the reply ends.

They are joined **in one go**, not glued on one at a time — with a whole file arriving in pieces, that is the difference between instant and painfully slow.

Why the fake model is the best decision in the project

Nearly every bug here is a **loop bug**: a request with no matching result, a counter that never resets, messages in the wrong order. You cannot chase those against a real model — it answers differently every time, and slowly. Against a script they are ordinary tests that run in **milliseconds**.

BUILD THIS SEAM ON DAY ONE

Ten minutes at the start: never call the model directly, always go through a swappable piece. Adding it *later*, once a dozen places call the model, is miserable.

And the privacy part

Because this is the only place that opens a connection, it is also the only place privacy has to be enforced:

- The address **must** be your own machine, pinned to the exact number it resolved to, so it cannot quietly become something else later.
- Proxy settings are **ignored**, so traffic meant for your own machine can never be routed out through someone else's server.
- If any of that can't be guaranteed, it **refuses to start**.

ONE BOX, ONE RESPONSIBILITY

All of this follows from a single sentence: **this is the only code that talks to the outside**. The translator, the fake model and the privacy lock are all about that one boundary.

07 What happens when you press Enter

THE MAIN FLOW



WHY IT LOOPS

One question and one answer cannot fix a bug. The model has to **look, try something, see what happened, then try again**. Every time round, it knows more than it did before.

WHAT MAKES IT STOP

An answer with **no requests in it**. If the model just talks, it means it has nothing left to look up — so what it said *is* the answer, and we stop.

HOW IT STAYS UNDER CONTROL

A hard limit of **80 times round**, plus eight more stop-conditions. Anything that writes a file or runs a command **asks you first**. Just reading? It goes ahead.

08 The whole agent, in 20 lines

Take away 3,000 lines of safety checks and polish, and this is the whole thing. It is not pseudo-code — this runs, and we tested it with a fake model.

the heart of every coding assistant ever built

```
def agent(task, max_steps=12):
    messages = [{"role": "system", "content": SYSTEM},
                {"role": "user", "content": task}]

    for _ in range(max_steps):          # the iteration guard
        reply = client.chat.completions.create(
            model=MODEL, messages=messages, tools=SCHEMA
        ).choices[0].message
        messages.append(reply.model_dump(exclude_none=True))

        if not reply.tool_calls:        # plain words = DONE
            return reply.content

        for call in reply.tool_calls:
            args = json.loads(call.function.arguments)
            if call.function.name in NEEDS_OK and not ask_user():
                out = "DENIED by the user."
            else:
                try:
                    out = TOOLS[call.function.name](**args)
                except Exception as e:    # errors are DATA
                    out = f"ERROR: {type(e).__name__}: {e}"
            messages.append({"role": "tool",
                            "tool_call_id": call.id,
                            "content": str(out)[:4000]})    # ALWAYS cap

    return "Stopped: hit the step limit."
```

Five small decisions, each a big deal

THE LINE	WHY IT HAS TO BE THERE
<code>str(out)[:4000]</code>	Tool output is added to the conversation and re-sent forever after. One giant result ruins the rest of the session. So we cut it short right where it is produced .
<code>except → "ERROR: ..."</code>	If a tool crashes, the whole thing stops. Instead we turn the failure into a message — and the model reads it and tries something else.
<code>range(max_steps)</code>	Without a limit, a confused model keeps trying forever. Your laptop runs all night.
<code>NEEDS_OK</code>	Anything that could break something asks you first. You stay in charge.
The system prompt	We change how it behaves by writing English , not code. Adding "look before you answer" genuinely changes the quality of the result.

THE ERROR MESSAGE IS THE IMPORTANT ONE

Compare two ways of reporting the same failure:

- "error" → the model has nothing to go on. It guesses, and keeps guessing wrong.
- "path not found; did you mean llmcode/agent.py?" → fixed on the very next try.

Your error messages are now instructions to the model.

Write them for the model to read, not for a log file nobody opens.

09 Tools — giving the model hands

TOOLS.PY

A TOOL IS JUST THREE THINGS

llmcode/tools.py — the Tool dataclass

```
Tool(
    name="read_file",
    description="Read a text file. Use before answering about code.",
    parameters={"type": "object", "required": ["path"], # JSON Schema →
                "properties": {"path": {"type": "string"}, # sent to the model
                              "offset": {"type": "integer"},
                              "limit": {"type": "integer"}},
    fn=_read_file, # the Python that runs
    requires_confirmation=False, # reads are safe
)
```

1. a description the model reads · 2. the Python that actually runs · 3. whether to ask you first.

EVERY TOOL ANSWERS THE SAME WAY

Either {"ok": true, ...} or {"ok": false, "error": ...} — never a crash. A crash would end your turn. An error just tells the model what went wrong, so it can try something else.

THE DESCRIPTION IS DOING REAL WORK

Write "Read a file" and the model reads whole files. Write "Read a file — for big ones, ask for a range" and it starts asking for ranges.

Same code underneath. Completely different behaviour.

The eleven tools

TOOL	WHAT IT DOES, AND WHAT PROTECTS YOU
<code>read_file</code>	Can read just part of a file. Refuses to dump huge generated files.
<code>write_file</code>	Asks first. Can only write inside your project folder.
<code>edit_file</code>	Swaps one exact piece of text. Forgives small spacing differences.
<code>run_bash</code>	Asks first. Gives up if it hangs. Cuts huge output. Your API keys are never passed to it.
<code>glob / grep</code>	Find files by name / find text inside them. Both stop at a sensible number of results.
<code>repo_map</code>	A short map of the project: files and what they contain.
<code>code_search</code>	Search your code by words, or by meaning.
<code>web_fetch</code>	Fetch a web page — but it is blocked from reaching anything inside your own network.
<code>spawn_agent</code>	Hand a job to a separate helper assistant.
<code>todo_write</code>	Lets the model keep its own to-do list.

TWO REAL BUGS THIS COST US

- Our own docs said writes couldn't escape the project folder. For `run_bash` that was **simply not true**. A safety promise that isn't real is worse than no promise, because people relax.
- Writing a file whose *contents* looked like a tool request corrupted it — the reader got confused about where the file ended.

10 Custom #1 — every model speaks a different language

PROVIDERS.PY

WHY

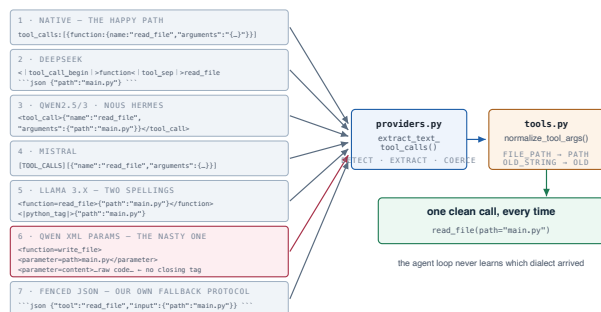
Cloud models all ask for things the same way. Local models come from a dozen labs, each with its own idea — and many **don't use the proper channel at all**. They type the request out as ordinary text and hope you notice.

WHAT

A reader that understands **seven ways of saying the same thing**: one proper format, five that model families made up, and one we invented for models that can do none of them.

HOW

Spot the style, pull out the name and details, turn it into one plain Python value. **The rest of the program only ever sees one shape.**



THE BUG THAT FORCED THIS

Writing files did not work for *weeks*. Always the same error: *"requires a string 'path'"*.

Two problems stacked up. The model wrote its request in a style we couldn't read yet — **and** it called the option `file_path`, because that is what it saw most in training, while our tool called it `path`.

THE GENUINELY HARD PART: WHERE DOES THE TEXT STOP?

A model will often say "here comes the file content", paste an entire Python file, and then **never say where it ended**. So we have to guess the end point.

Guess too early and half the file is lost. Guess too late and **the next request gets written into the user's source file**. That second one actually happened.

11 Custom #2 — wrong argument names, and refusing to guess

TOOLS.PY

Even once we can read the request, the **option names are often wrong**. Local models learned from other tools' docs, so they use *those* names — `file_path` where we say `path`.

llmcode/tools.py — the alias table (excerpt)

```
_ARG_ALIASES = {
    "read_file": {
        "path": ("file_path", "filepath", "filename", "file", "target_file")},
    "write_file": {
        "path": ("file_path", "filepath", ...),
        # NOTE: deliberately NO new_string/new_str here — those are edit_file's
        # `new` key; mapping them to write_file content would turn a mis-targeted
        # edit into a DESTRUCTIVE whole-file overwrite with just the fragment.
        "content": ("file_text", "text", "body", "data", "code")},
    "edit_file": {
        "old": ("old_string", "old_str", "search", "find"),
        "new": ("new_string", "new_str", "replace")},
}
```

READ THAT COMMENT IN THE MIDDLE AGAIN

This table **guesses what the model meant**. One obvious-looking guess was deliberately left out, because getting it wrong would **overwrite a whole file with a two-line fragment**. Knowing which guesses *not* to make is the whole skill.

The rule that keeps the guessing safe

```
for canonical, syns in aliases.items():
    cur = args.get(canonical)
    if cur not in (None, ""):
        continue # model gave a real value — authoritative
    if cur == "":
        continue # explicit empty — intentional, keep it
    for syn in syns: # else backfill from the first alias present
        ...
```

- **A guess can only fill in a blank.** It can never overrule something the model actually said.
- **"Nothing" is a real answer.** Empty content means "make an empty file" — that is an instruction, not a gap to fill.
- **Nothing is touched unless we actually fix something** — so what you were shown in the confirmation prompt is exactly what runs.

And two places we deliberately refuse to guess

WHAT HAPPENED	WHAT WE DO ABOUT IT
Two tool-shaped blocks in one reply	Run none of them. If we ran only the first, the second would vanish silently and the model would never find out — it would carry on believing both had happened.
A block inside prose or a <code><think></code> section	Ignore it. Otherwise, a model <i>explaining</i> how <code>rm -rf</code> works would actually delete your files.

THE RULE WORTH TAKING AWAY

Be generous about what you try to understand. Be strict about what you actually do.

Misunderstanding costs one wasted turn. Acting on a misunderstanding costs the user's files.

12 Keeping the chat small — why not just send everything?

BOX 5 OF 10 · ORANGE, TOP LEFT

First, the thing that makes this necessary

The model has **no memory between messages**. So every single turn, we send the *entire* conversation again from the beginning — and it reads all of it before writing one word of the reply.

Imagine a colleague who forgets everything between sentences and has to re-read the whole meeting transcript each time you speak. That is exactly what is happening.

So why not just send it all and let it sort it out?

Three reasons, and only the first is the obvious one:

REASON	WHAT ACTUALLY HAPPENS
1. It gets slow	Reading time grows with the chat. Typing speed doesn't change — so it just sits in silence for longer and longer before it starts answering.
2. It doesn't fit	A local model can only hold so much. Past that point the oldest messages fall off silently — including, quite possibly, the instruction that mattered most.
3. The answers get worse	This is the one people miss. Your actual question ends up buried in thousands of lines of stale tool output, and the model starts answering about a file it read six steps ago. More context is not more understanding.

THE MEASUREMENT THAT STARTED ALL THIS

Someone typed "*what happened?*" — four words. We sent **55,000 words** of old conversation with it. The reply took **about 20 minutes**.

Over half of what we sent was old tool output — files read six steps earlier that nobody needed any more.

So how do we pick just the part that's needed?

Five steps, run in this order at the start of every turn — cheapest first, and each one only does something if the one before wasn't enough:

#	STEP	WHAT IT DECIDES
1	Judge the question	Quick question, or real work? "thanks" and "refactor the parser" should not get the same treatment.
2	Set a size budget	A number for <i>this</i> turn. Small for a quick question, up to 3× for a whole-project job.
3	Shorten what's stale	Tool results from <i>finished</i> steps get cut to their first few lines, with a note saying how to get the rest back.
4	Look up what's relevant	Search past conversations for the few bits that match this question, and attach only those. (box 7 — RAG)
5	Summarise, if still too big	Ask the model to condense the old turns into one short note. This costs a round trip, so it's the last resort.

THE RULE UNDERNEATH ALL FIVE

The most recent exchange is always kept whole.

Everything being shortened or summarised is from turns that are already finished. What you just said, and what just happened, is never touched.

55k → 5k

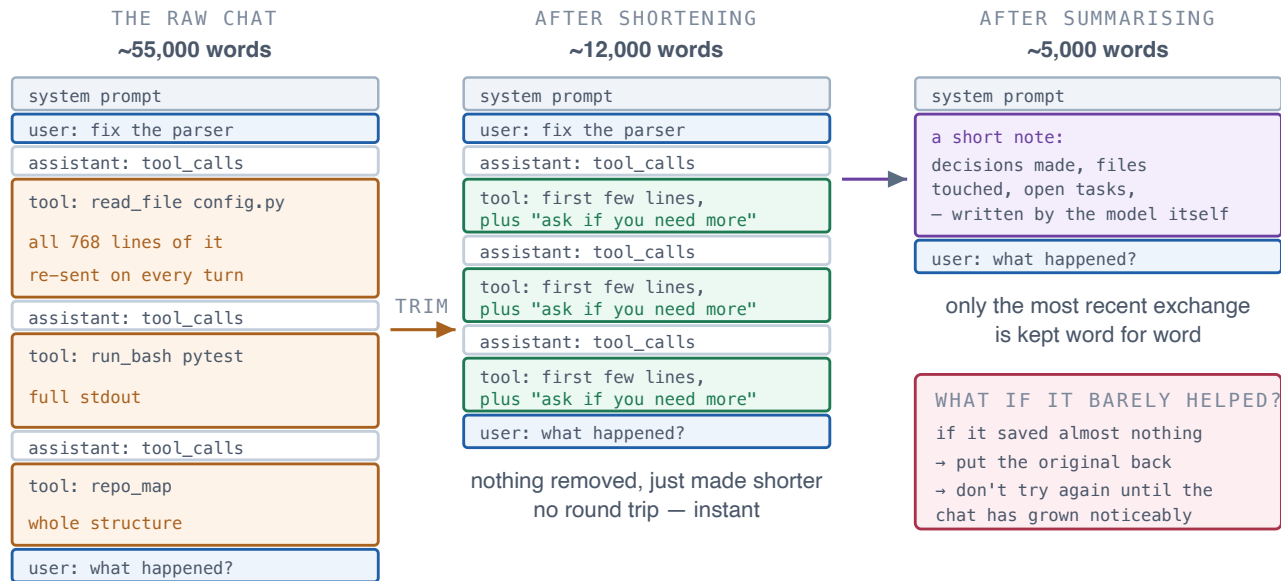
words sent, same question

20 min → 24 s

time to answer, same laptop

13 Keeping the chat small — how the shortening works

BOX 5 OF 10 · CONTINUED



SHORTENING — FREE, AND IT HAPPENS EVERY TURN

Old tool results get cut to their first few lines. A note is left saying *"there was more here — ask again if you need it."*

Two details matter. The note **explains itself**, so the model re-reads the file rather than inventing what was in it. And anything already shortened is **left alone next time** — otherwise each turn would nibble off another piece until the note ate itself.

SUMMARISING — COSTS A ROUND TRIP

When shortening isn't enough, we ask the model to write **one short note** covering the old turns: decisions made, files touched, what's still to do.

This is **not** deleting old messages. Deleting is one line of code — and it throws away every decision you made together.

SUMMARISING CAN BACKFIRE — SO WE CHECK

A small model sometimes writes a summary **as long as the thing it summarised**.

So we keep a copy first. If it didn't save at least a few percent, we **put the original back**. A fix that doesn't fix anything still costs you time.

14 Re-using the server's work — free speed

BOX 6 OF 10 · PURPLE, LEFT SIDE

WHY THIS WORKS

The server **remembers the reading it already did**. If your next message starts with exactly the same text as the last one, it can skip straight past all of it — which is most of your waiting time, for free. The industry name for this is prompt caching, or **CAG**.

WHAT WE DO

One rule, everywhere in the code: **only ever add to the end of the conversation**. Never insert in the middle. Never reorder. Never edit what came before.

HOW THE EXTRAS FIT IN

Anything temporary — things we looked up for this question, a hint for this message — is stuck on the **very end**, and only on the first try. The end was never cached anyway, so it costs nothing.

✓ ADD IT TO THE END — WHAT WE DO



x SQUEEZE IT INTO THE MIDDLE — THE TEMPTING MISTAKE



One message added near the front throws away the saved work for everything after it — even though not one of those words changed. Where it sits matters, not just what it says.

BOTH PICTURES CONTAIN THE SAME INFORMATION

The only difference is **where the new note was put**. Put it at the end and the server skips almost everything. Put it in the middle and the server has to re-read every word after it — even though none of those words changed.

THE SAME RULE SHOWS UP THREE MORE TIMES

- A message with no picture stays plain text, because attaching one changes its shape.
- The tool descriptions are built once and re-used, so their order can't drift.
- A summary that didn't help gets undone — the code would rather carry extra words than disturb the beginning.

15 Looking things up — remembering without re-reading

BOX 7 OF 10 · BLUE, RIGHT SIDE

WHY

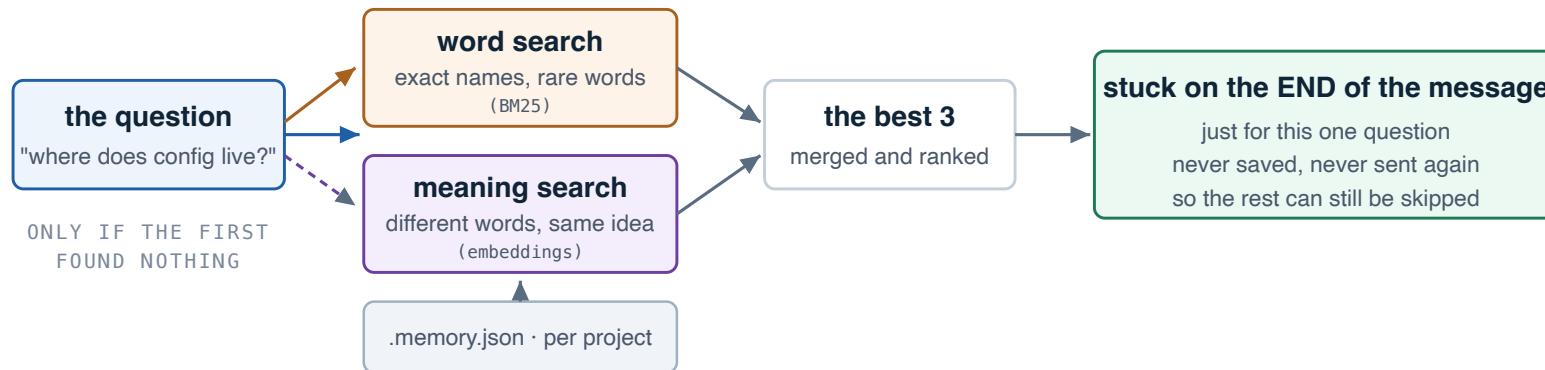
We just said: send less history. But some of it genuinely matters. Drop it and the assistant is forgetful; keep it all and it is slow.

WHAT

Keep everything in a **store**. **Search** it with whatever was just asked. Send only the **few pieces that matched**. The usual name for this is **RAG**.

HOW

Two search engines, both written by hand: one for **exact words**, one for **meaning**. The results are merged and attached as a single note.



WHERE IT LIVES

One plain file per project, with the numbers in a second file beside it. The index is built once and re-used — otherwise every search would re-read everything.

RETRIEVAL RUNS ONCE PER TURN

Before the loop starts, never inside it. Searching costs time; doing it on every step would undo the speed it was meant to buy. A message like "thanks" skips it completely.

FAILURE IS ALWAYS SWALLOWED

If the search breaks for any reason, we carry on with nothing. Memory is a **bonus**. Losing it must never cost you the answer.

1 · RARE WORDS COUNT FOR MORE

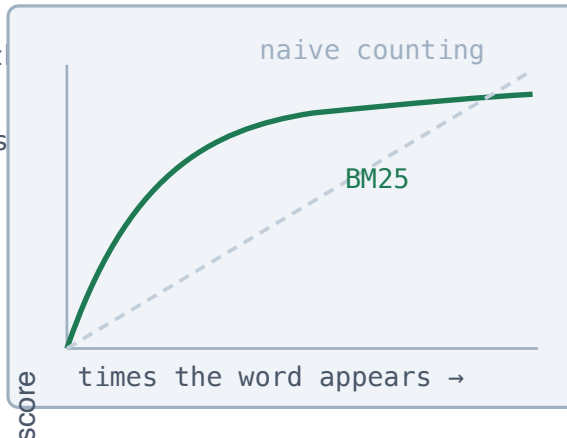
"the"  appears in everything → worthless

"config"  appears in a few → useful

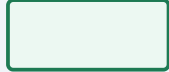
"_maybe_a"  appears in exactly one → gold

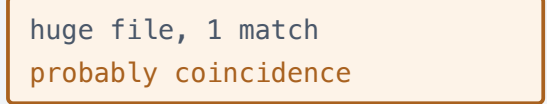
A word that is in everything tells you nothing.

2 · REPEATS COUNT LESS AND LESS



3 · LONG DOCUMENTS GET A PENALTY

 short note, 1 match
strong signal

 huge file, 1 match
probably coincidence

Finding a word in 20 lines means more than finding it in 2,000.

llmcode/memory.py — the whole scoring loop, nothing hidden

```
for term in q_terms:
    f = tf.get(term, 0)          # times the term is in THIS doc
    if f == 0: continue
    n_q = df[term]              # how many docs have it at all
    idf = math.log(1 + (n - n_q + .5) / (n_q + .5)) # rarity
    denom = f + k1 * (1 - b + b * (dl / avgdl)) # length
    s += idf * (f * (k1 + 1)) / denom # saturating
```

Why not just count how many words match? Because every document contains "the". This formula is the classic fix. It still beats fancier methods on **code**, where an exact function name matters far more than general vibes. Its proper name is **BM25**.

THROWING AWAY "THE" AND "AND" IS DOING REAL WORK

It looks like tidying. It isn't. If a question and an old note share *only* the word "the", the score comes out **above zero** — and the next slide only reaches for the meaning search when the score is **exactly zero**.

Removing filler words is what makes "zero" **actually mean "nothing in common"**.

DON'T COPY A FILLER-WORD LIST OFF THE INTERNET

Ours deliberately **keeps** on, off, no and not. In a command-line tool those aren't filler — they're *values*, as in `/mcp on`.

A list copied from somewhere else would have quietly deleted them, and searches for commands would have stopped working.

WHAT "SEARCHING BY MEANING" ACTUALLY IS

Every piece of text is turned into a **list of 768 numbers**. Think of it as a position on a map, where things about the same topic end up near each other.

"where do my settings live?" lands right next to "the config file is in ~/.llmcode" — even though they share **not one word**.

To compare two of these, we measure the **direction** they point in, not how far away they are — distance mostly just reflects how wordy the text was. That is 12 lines of code:

llmcode/memory.py — cosine()

```
dot = na = nb = 0.0
for x, y in zip(a, b):
    dot += x * y      # how much they point the same way
    na += x * x
    nb += y * y
if na <= 0.0 or nb <= 0.0: return 0.0
return dot / math.sqrt(na * nb)    # 1.0 = identical meaning
```

NOTICE WHAT IT PROMISES

It **never crashes**. Given nonsense it returns zero — which means "no match", and is exactly the right answer. One corrupted note can never take down your whole turn.

The one decision that made this fast

Running both every time means an extra round trip on every turn — and on one GPU that can force the whole model to be swapped out and back. So:

llmcode/memory.py — the entire policy, in one condition

```
best_bm25 = bm25_full[0][1] if bm25_full else 0.0
if provider is not None and (
    mode == "embed" or (mode == "auto" and best_bm25 <= 0.0)
):
    embed_idx = self._embed_rank(query, provider, pool_k)
```

CASE	WHAT RUNS	COST
Shares real words "where is config.json?"	word search only	instant — and this is the usual case
A paraphrase "where do settings live?"	word search finds nothing → try meaning search → merge	one extra round trip, only when needed
Trivial message "thanks"	nothing	zero

THE PATTERN TO STEAL

Do the cheap check first. Only pay for the expensive one when the cheap one found nothing. The same shape turns up four times in this codebase.

WHY

Those lists of numbers must survive closing the terminal. The usual answer is a special database — often a whole extra program running in the background. Absurd for a personal tool holding a few thousand items.

WHAT

A plain list in memory, one readable file for the text, and a **second file holding just the raw numbers**. Searching is an ordinary loop.

HOW

Each number takes **4 bytes** instead of the ~32 an ordinary Python number costs. And saving them as raw bytes avoids turning every single one into text and back again.

IN RAM

```
{
  "a7f3...": array('f', [0.13, -0.88, 0.04, ...]),
  "b0c1...": array('f', [0.52, 0.19, -0.7, ...])
}
```

4 bytes per number — not Python's bulky floats



SAVE

ON DISK — TWO FILES

.memory.json

the text, plus a fingerprint of the numbers
readable, and small

.memory.json.vec

just the raw numbers, back to back
all the same size — no labels, no punctuation

THE .VEC BYTE LAYOUT

vector 0 · 3072 B vector 1 · 3072 B vector 2 · 3072 B ...

same order as the list in the other file, so number 5 is easy to find

WHAT IF THE POWER CUTS HALFWAY?

Two files can disagree. If we save one and not the other, one says 200 items and the other holds 140 — and every item quietly matches the **WRONG** numbers.

Fix: save a fingerprint of the numbers file inside the other file. Check it on load.

If it doesn't match → throw the numbers away and rebuild them. Slow once, never wrong.

WHEN THIS SIMPLE APPROACH IS THE RIGHT ONE

Thousands of items, one machine, one person. Looping over 4,000 of them takes **milliseconds**.

You need a real database at *millions* of items, or when several programs share one index — **not before**.

THE LESSON WORTH KEEPING

Whenever **two files have to agree with each other**, one of them must be able to **prove** it matches. A short fingerprint is the cheapest possible proof.

It is the difference between "rebuild once on next launch" and "quietly wrong answers, forever, with no error".

WHY

A small model cannot read a 50-file project. Three files already fill it up. It needs to **find the right 40 lines**, not read everything and hope.

WHAT

A search the model runs **itself**, using the same two engines as memory — plus a short map of the project so it knows what exists at all.

HOW

The code is cut up **function by function**, not every 50 lines — so every result is a whole, complete thing. Built once, re-used, and always cut short before it gets huge.

Three deliberate choices

CHOICE	REASON
AST-aware chunking	A result that starts halfway through a function teaches the model nothing. Where you cut should match where the <i>meaning</i> ends.
BM25 by default embeddings opt-in via <code>/codeembed</code>	Searching code usually means searching for a name you already know. Building a meaning index takes minutes on a laptop; the word search takes milliseconds and usually wins anyway.
Its own output cap smaller than every other tool's	One search can return a lot. Cutting it shorter than a file read stops a single search from eating the entire turn.

The repo map: which files even matter?

A flat list of 300 filenames tells the model nothing. So we rank them, using the same idea Google originally used to rank web pages — applied to **which file uses which**.

THE INTUITION

A file that many other files depend on is **probably important** — just as a web page that many pages link to probably matters. `config.py` scores high; a one-off script scores low.

Then: fit it to a budget

- The map has to fit a size limit — but you can't know how many entries fit until you actually write them out.
- So we **guess, measure, halve the gap, repeat** — about ten tries to land exactly on the limit. (The same trick as guessing a number between 1 and 1000.)
- The ranking itself had to be rewritten — the obvious version re-checked every connection for every file, and made this the slowest tool we had.

THE PATTERN BEHIND ALL THREE CHAPTERS OF RAG

Don't give the model more to read. Give it better ways to find what it needs.

A good search tool beats a bigger memory, and costs far less.

WHY

"Which file handles settings?" might take twenty file reads to answer. Do that in the main conversation and you have filled it with files that turned out not to matter — and every later turn re-sends them all.

WHAT

A tool that starts a **second assistant** with its own private conversation and a **limited set of abilities**. It answers in a few lines. Its twenty file reads are **thrown away with it**.

HOW

Start a fresh assistant with its own instructions and tools, let it finish, and hand its final answer back as if it were any other tool result.

THE TRADE IN NUMBERS

The helper reads 20 files → **40,000 words in its conversation**.

The main assistant gets back **about 15 lines**.

The main conversation stays small **because of how it is built**, not because the model remembered to be careful.

Four roles, different tools

ROLE	TOOLS	JOB
orchestrator	all + <code>spawn_agent</code>	Plan the work, hand out jobs, check the result
explorer	read-only	Look around and report back. Physically cannot break anything
coder	full	Make one specific change, then check it worked
reviewer	read-only	Find problems before we call it finished

SAFETY BY SIMPLY NOT GIVING IT THE TOOL

The explorer **has no way to write files at all**. No clever wording in a file it reads can trick it into deleting something, because the ability isn't there.

Take the tool away. Don't ask the model nicely.

The same idea, scaled up: `/audit`

- Reviewing a whole project doesn't fit in a local model's head. Not close.
- So we **split it up**: review each small piece with a fresh helper, then merge all the findings into one report.
- Each helper stays small and fast — the exact opposite of one giant request.

GENERALISABLE

When a job doesn't fit, **don't go looking for a bigger model — split the job**.

WHY

A cloud tool never thinks about hardware. On **your** laptop, an hour of flat-out work means screaming fans — then the machine **deliberately slows itself down** to cool off. The longer you work, the worse it gets.

WHAT

Two features **no cloud tool has**:

- a forced **one-minute break** after roughly ten minutes of solid work
- **shorter replies and small pauses** between turns

HOW

The break happens at the **top of the loop**, where nothing is half-finished. Both are on by default, and both switch themselves off when output is piped — so scripts and tests never sit there waiting.

WHY THE TOP OF THE LOOP

Pausing in the middle of a step would leave a request with no matching result sitting in the conversation — and most servers **reject the whole thing** when they see that.

The position isn't arbitrary — it makes breaking **impossible**, not just unlikely. Ctrl-C is handled at the same spot, for the same reason.

THE BUG THAT GENTLE MODE CAUSED

We capped how much the model could write, to keep the GPU cool. On a normal answer, hitting the cap just cuts off a sentence.

But when it *writes a file*, the file content travels **inside the request itself**. Cut that off and the request is incomplete — the model thought it had written the file, the user thought so too, and **nothing reached the disk**.

The fix, in three parts: the cap is now **skipped entirely whenever tools are on the table**; the default was raised **1,024 → 4,096**; and anything still cut off is retried **once** with no cap at all.

And the twist: cutting off then retrying meant the model wrote the answer *twice* — which produced **more** heat than the cap ever saved.

THE LESSON

A limit you set for one reason will be hit by something you never thought about. Whenever you add a cap, ask what *e/else* travels down that path.

THE HONEST REFRAKE

We spent a long time trying to make it *type* faster. Typing speed was **never** the problem — it sat near 95 words a second throughout. The real pain was the **waiting**.

Nobody experiences "words per second". They experience "**I pressed Enter, and then I waited.**" That is the number to improve.

Nine ways it knows to stop

WHAT IT WATCHES FOR	THE LIMIT
Going round and round	80 times, then stop
Requests we can't read at all	3 bad rounds
Asking for the exact same thing again	4 repeats
It thought, but said nothing	2 retries
The reply got cut off	once per round
The request was malformed	once each
You pressed Ctrl-C	checked every round
It's reading far too much	nudged once a turn

A SAFETY NET THAT CATCHES HEALTHY BEHAVIOUR IS A BUG

Our first version counted **any** bad request and shut down sessions that were working fine. A model that makes one good request and one bad one *is still getting somewhere*.

Now it only counts when **everything** in that round failed — and a single good request resets the count.

Treat the model like any untrusted input

- **It asks you first** before writing, editing, or running anything.
- **Workspace confinement** on the *file* tools (read / write / edit / glob / grep / repo_map) — paths resolving outside the launch directory are refused.
- **Your secrets are stripped out** — commands it runs never inherit your API keys or cloud credentials.
- **Web fetching can't reach inside your network** — private addresses are refused, and re-checked on *every redirect*, so it can't be talked into it halfway.
- **Undo checkpoints** — files are snapshotted before every edit.
- **Small mistakes fix themselves** — a wrong path is corrected and retried, a limited number of times.

--PRIVATE : AN ENFORCED OFFLINE LOCKDOWN

Not a promise — an actual mechanism. The model address **must** be your own machine. Web fetching is **removed entirely**, so the model can't even ask. Commands run with the network blocked, and if that blocking isn't available, they **refuse to run at all** rather than run unprotected.

THE MOST VALUABLE BUG WE EVER FIXED

Our own docs said the tool couldn't write outside your project folder. For shell commands that was simply **not true** — it is a raw shell, with no path check in it at all. What actually protects you there is **the confirmation prompt**.

A safety claim that isn't true is worse than no claim, because people relax *because* of it. Say which mechanism enforces a guarantee — or don't make it.

23 Custom #14 — other people's tools, and testing without a GPU

MCP.PY ·
PROVIDERS.PY

Plugging in tools we didn't write

WHY

Our eleven tools can't cover everything. Other people have already built tools for databases, note vaults, ticket systems. **MCP** is an agreed way for any AI tool to talk to any of them, so nobody has to write a special connector each time.

WHAT

741 lines, no new libraries. Each tool program is started as a small background process, and its abilities are added to the model's list next to our own.

HOW

Start it, ask what it can do, add those to the list. One that fails to start is **skipped with a warning** — it never takes the whole program down with it.

NEVER TRUST A SERVER'S OWN HINTS

A tool program can declare "this one is harmless, no need to ask". But a *bad* one could say that about something destructive, to get it run automatically.

So that claim is ignored unless **you** have said you trust that particular program.

The decision to trust is always yours, never the tool's.

Also: names are checked so one program can't impersonate another, duplicates are skipped rather than silently overwriting, and a program that stops responding is **shut down** — so a late reply can never be mistaken for the answer to a different question.

The single best decision in this project

We never let the code call the model directly. It always goes through a **swappable piece**. That let us write a second one: a **fake model that follows a script**.

real or fake, both produce exactly these three things

```
{"type": "text",      "text": <incremental delta>}
{"type": "tool_call", "id", "name", "arguments": <dict>}
{"type": "done",      "finish_reason", "output_tokens"} # exactly
one, last
```

1,355

tests

0

of them need
a GPU or internet

3

dependencies
openai · rich · prompt_toolkit

WHY THIS MATTERS MORE THAN IT SOUNDS

Nearly every bug here is a **loop bug**: a request with no matching result, a counter that never resets, messages added in the wrong order.

You cannot chase those with a real model — it gives a different answer every time, and slowly. Against a script they become **ordinary tests that run in milliseconds**.

DO THIS ON DAY ONE

Adding this swappable piece later, once twelve different places all call the model directly, is miserable work. Done at the start, it costs ten minutes.

What we built, in one line each

THE PART	THE IDEA BEHIND IT
The loop	Let the model ask for things. Do them. Show it what happened. Repeat until it just answers.
Tools	Always cut output short, never crash, and write error messages <i>for the model to read</i> .
Dialect parsing	Understand seven ways of asking — but refuse to act when it's unclear.
Context mgmt	Measure, budget, shorten, summarise. Send what <i>this</i> question needs, nothing more.
CAG	Only ever add to the end, so the server can skip re-reading. Free speed.
RAG	Search a store instead of pasting everything in. Cheap check first, expensive one only if needed.
Vector store	Raw numbers in a second file, plus a fingerprint so a crash can't scramble them.
Sub-agents	When a job doesn't fit, split the job.
Hardware	Breaks and shorter replies — the laptop is part of the system too.
Safety	Treat the model as untrusted. Ask first, fence it in, strip secrets, keep an undo.

IF YOU REMEMBER ONE THING

The model is not the program. Around 95% of this code is the machinery *around* the model — and that machinery is where the quality actually comes from.

Start here: a working assistant in 20 lines

Everything here grew out of the loop on slide 4. Type it, run it, then grow it — each exercise rebuilds one slide:

#	EXERCISE	WHAT YOU'LL DISCOVER
1	Add writing and editing files	Edits fail over invisible spacing. You'll invent a forgiving match.
2	Print the conversation size after each turn	Watch it explode after three file reads. That's the whole problem.
3	Summarise old turns once it gets big	And why simply deleting them loses your decisions.
4	Build a word search over your code	It beats meaning-search on code far more often than you'd think.
5	Add meaning-search, but only when the word search finds nothing	Cheap first, expensive only when needed.
6	Copy files before editing; add an undo	People approve far more changes once undo exists.
7	Send a read-only helper to do the exploring	Its reading gets thrown away; yours stays clean.
8	Point it at a model that asks in a different format	Reality. You'll write your second reader, then your third.

FINAL ADVICE

Build the loop first, and let it be ugly. Then spend your time on **better tools** and **keeping the conversation small**. That is where almost all the difference between a frustrating assistant and a genuinely useful one actually lives.